RDF 1.2 Semantics

W3C Working Draft 01 February 2026



▼ **More details about this document**

This version:

https://www.w3.org/TR/2026/WD-rdf12-semantics-20260201/

Latest published version:

https://www.w3.org/TR/rdf12-semantics/

Latest editor's draft:

https://w3c.github.io/rdf-semantics/spec/

History:

https://www.w3.org/standards/history/rdf12-semantics/

Commit history

Test suite:

https://w3c.github.io/rdf-tests/rdf/rdf12/

Latest Recommendation:

https://www.w3.org/TR/rdf11-mt

Editors:

Peter Patel-Schneider

Dörthe Arndt

Enrico Franconi

Former editor:

Patrick J. Hayes

Feedback:

GitHub w3c/rdf-semantics (pull requests, new issue, open issues)

public-rdf-star-wg@w3.org with subject line `[rdf12-semantics] … message topic …` (archives)

Copyright © 2004-2026 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

This document describes a precise semantics for *RDF 1.2 Concepts and Abstract Data Model* [*RDF12-CONCEPTS*] and *RDF 1.2 Schema* [*RDF12-SCHEMA*]. It defines a number of distinct entailment regimes and corresponding patterns of entailment. It is part of a suite of documents which comprise the full specification of RDF 1.2.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document is part of the RDF 1.2 document suite. This is a revision of the 2014 Semantics specification for RDF [*RDF11-MT*] and supersedes that document.

This document was published by the RDF & SPARQL Working Group as a Working Draft using the Recommendation track.

Publication as a Working Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress. Future updates to this upcoming Recommendation may incorporate new features.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document
Set of Documents

Notes

1. Introduction

2. Conformance

3. Semantic Extensions and Entailment Regimes

4. Notation and Terminology
4.1 Shared blank nodes, unions, and merges

5. Simple Interpretations
5.1 Blank nodes
5.1.1 Shared blank nodes
5.2 Simple Entailment
5.3 Properties of simple entailment and satisfiability

6. Skolemization

7. Literals and datatypes
7.1 D-interpretations
7.2 Datatype entailment
7.2.1 Patterns of datatype entailment

8. RDF Interpretations

8.1 RDF entailment
8.1.1 Patterns of RDF entailment

9. RDFS Interpretations
9.1 A note on rdfs:Literal
9.2 RDFS entailment
9.2.1 Patterns of RDFS entailment

10. RDF Datasets

11. Appendices

A. Entailment rules

B. Proofs of some results

C. Semantic Adequacy Considerations

D. Privacy Considerations

E. Security Considerations

F. Acknowledgments

G. Substantive Changes
G.1 Substantive changes between RDF 1.0 and RDF 1.1
G.2 Substantive changes since RDF 1.1

H. Index
H.1 Terms defined by this specification
H.2 Terms defined by reference

I. References
I.1 Normative references
I.2 Informative references

§ Notes

Technical Note

Notes in this style are technical asides on obscure or recondite matters.

§ 1. Introduction

This document defines a model-theoretic semantics for RDF graphs and the RDF and RDFS vocabularies, providing an exact formal specification of when truth is preserved by transformations of RDF or operations which derive RDF content from other RDF.

§ 2. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MAY*, *MUST*, *MUST NOT*, *SHOULD*, and *SHOULD NOT* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

This specification, *RDF 1.2 Semantics*, is normative for RDF semantics and the validity of RDF inference processes. It is not normative for many aspects of RDF meaning which are not described or specified by this semantics, including social issues of how IRIs are assigned meanings in use and how the referents of IRIs are related to Web content expressed in other media such as natural language texts.

§ 3. Semantic Extensions and Entailment Regimes

RDF is intended for use as a base notation for a variety of extended notations such as OWL [*OWL2-OVERVIEW*] and RIF [*RIF-OVERVIEW*], whose expressions can be encoded as RDF graphs which use a particular vocabulary with a specially defined meaning. Also, particular IRI vocabularies may be given meanings by other specifications or conventions. When such extra meanings are assumed, a given RDF graph may support more extensive entailments than are sanctioned by the basic RDF semantics. In general, the more assumptions that are made about the meanings of IRIs in an RDF graph, the more entailments follow from those assumptions.

A particular such set of semantic assumptions is called a ***semantic extension***. Each semantic extension defines an ***entailment regime*** (used here in the same sense as in the *SPARQL 1.2 Entailment Regimes* recommendation [*SPARQL12-ENTAILMENT*]) of entailments which are valid under that extension. RDFS, described later in this document, is one such semantic extension. We will refer to entailment regimes by names such as *RDFS entailment*, *D-entailment*, etc.

Semantic extensions *MAY* impose special syntactic conditions or restrictions upon RDF graphs, such as requiring certain triples to be present, or prohibiting particular combinations of IRIs in triples, and *MAY* consider RDF graphs which do not conform to these conditions to be errors. For example, RDF statements of the form

```
ex:a rdfs:subClassOf "Thing"^^xsd:string .
```

are prohibited in the OWL semantic extension based on description logics [*OWL2-SYNTAX*]. In such cases, basic RDF operations such as taking a subset of triples, or combining RDF graphs, may cause syntax errors in parsers which recognize the extension conditions. None of the semantic extensions normatively defined in this document impose such syntactic restrictions on RDF graphs.

All entailment regimes *MUST* be ***monotonic*** extensions of the simple entailment regime described in the document, in the sense that if A simply entails B then A also entails B under any extended notion of entailment, provided that any syntactic conditions of the extension are also satisfied. Put another way, a semantic extension cannot "cancel" an entailment made by a weaker entailment regime, although it can treat the result as a syntax error.

§ 4. Notation and Terminology

This document uses the following terminology for describing RDF graph syntax, all as defined in the companion RDF Concepts specification [*RDF12-CONCEPTS*]: ***IRI***, ***RDF triple***, ***triple term***, ***RDF graph***, ***subject***, ***predicate***, ***object***, ***RDF source***, ***node***, ***blank node***, ***literal***, ***RDF term***, ***isomorphic***, ***appears in***, and ***RDF dataset***. All the definitions in this document apply unchanged to ***generalized RDF triples***, ***generalized RDF graphs***, and ***generalized RDF datasets***.

A ***ground RDF graph*** is an RDF graph in which no blank nodes appear. A ***ground RDF term*** is an RDF term in which no blank nodes appear. A ***ground triple term*** is a triple term term in which no blank nodes appear. A ***ground RDF triple*** is an RDF triple in which no blank nodes appear.

An ***interpretation*** is a mapping from ground RDF terms into a set, together with some constraints upon the set and the mapping. This document defines various notions of interpretation, each corresponding in a standard way to an entailment regime. These are identified by prefixes such as *simple interpretation*, etc., and are defined in later sections. The unqualified term *interpretation* is usually used to refer to any compatible kind of interpretation in general, but if clear from the context might refer to a specific kind of interpretation.

The word ***denotes*** is used here for the relationship between a ground RDF term and what it refers to in a given interpretation, itself called the ***referent***. (The phrase ***refer to*** is often used instead of denote and ***denotation*** instead of referent.) IRI meanings may also be determined by other constraints external to the RDF semantics; when we wish to refer to such an externally defined naming relationship, we will use the word ***identify*** and its cognates. For example, the fact that the IRI `http://www.w3.org/2001/XMLSchema#decimal` is widely used as the name of a datatype described in the XML Schema document [*XMLSCHEMA11-2*] might be described by saying that the IRI *identifies* that datatype. If an IRI identifies something it may or may not denote it in a given interpretation, depending on how the semantics is specified. For example, an IRI used as a graph name identifying a named graph in an RDF dataset may denote something different from the graph it identifies.

Throughout this document, the equality sign = indicates strict identity. The statement "A = B" means that there is one entity which both expressions "A" and "B" denote. Angle brackets < x,

y > are used to indicate an ordered pair of x and y.

Throughout this document, RDF graphs and their components are written using the notational conventions of the Turtle syntax [*RDF12-TURTLE*]. The namespace prefixes `rdf:` `rdfs:` and `xsd:` are used as in the RDF Concepts specification [*RDF12-CONCEPTS*], RDF vocabularies. When the exact IRI does not matter, the prefix `ex:` is used. When stating general rules or conditions we use three-character variables such as aaa, xxx, sss to indicate arbitrary IRIs, literals, or other components of RDF syntax. Some cases are illustrated by node-arc diagrams showing the graph structure directly.

A ***name*** is any IRI or literal. A literal contains two names: itself and its internal type IRI. A ***vocabulary*** is a set of names.

The ***empty graph*** is the empty set of triples.

A ***subgraph*** of an RDF graph is a subset of the triples in the graph. A triple is identified with the singleton set containing it, so that each triple in a graph is considered to be a subgraph. A ***proper subgraph*** is a proper subset of the triples in the graph.

For RDF terms t, x, and y, where either y is not a triple term or x does not appear in y, we define the ***substitution mapping*** t[x/y] inductively, as follows:

1. If t = x, then t[x/y] = y.
2. Otherwise, if t = (s, p, o), then t[x/y] = (s[x/y], p[x/y], o[x/y]).
3. Otherwise, t[x/y] = t.

For any triple (s, p, o), we define (s, p, o)[x/y] as (s[x/y], p[x/y], o[x/y]). For any graph G, we define G[x/y] as the result of applying [x/y] to each triple t in G.

Suppose that M is a functional mapping from a set of blank nodes to some set of RDF terms. Any graph obtained from a graph G by replacing some or all of the blank nodes N appearing in G by M(N) is an ***instance*** of G. Any graph is an instance of itself, an instance of an instance of G is an instance of G, and if H is an instance of G then every triple in H is an instance of at least one triple in G.

A ***proper instance*** of a graph is an instance in which a blank node has been mapped into something other than a blank node, or two blank nodes in the graph have been mapped into the same blank node.

Two graphs are isomorphic when each maps into the other by a 1:1 mapping on blank nodes. Isomorphic graphs are mutual instances with an invertible instance mapping. As blank nodes have no particular identity beyond their location in a graph, we will often treat isomorphic graphs as identical.

An RDF graph is ***lean*** if it has no instance which is a proper subgraph of itself. A ground RDF graph is lean. Non-lean graphs have internal redundancy and express the same content as their lean subgraphs. For example, the graph in Example 1 is not lean:

EXAMPLE 1: Non-lean graph

```
ex:a ex:p _:x .
_:y ex:p _:x .
```

In contrast, the graph in Example 2 *is* lean:

EXAMPLE 2: Lean graph

```
ex:a ex:p _:x .
_:x ex:p _:x .
```

§ 4.1 Shared blank nodes, unions, and merges

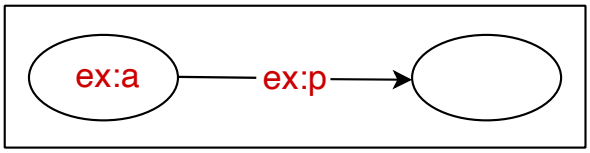
Graphs share blank nodes only if they are derived from graphs described by documents or other structures (such as an RDF dataset) that explicitly provide for the sharing of blank nodes between different RDF graphs. Simply downloading a web document does not mean that the blank nodes in a resulting RDF graph are the same as the blank nodes coming from other downloads of the same document or from the same RDF source.

RDF applications which manipulate concrete syntaxes for RDF which use blank node identifiers should take care to keep track of the identity of the blank nodes they identify. Blank node identifiers often have a local scope, so when RDF from different sources is combined,

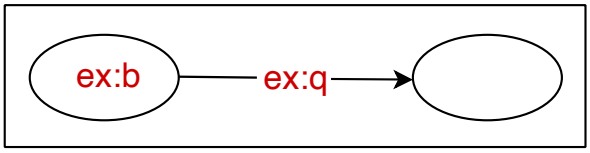
identifiers may have to be changed in order to avoid accidental conflation of distinct blank nodes.

For example, two documents may both use the blank node identifier "`_:x`" to identify a blank node, but unless these documents are in a shared identifier scope or are derived from a common source, the occurrences of "`_:x`" in one document will identify a different blank node than the occurrences of "`_:x`" in the other document. When graphs are formed by combining RDF from multiple sources, it may be necessary to ***standardize*** apart the blank node identifiers by replacing them with other identifiers which do not occur in the other document(s). For example, the two graphs for the two texts below contain two nodes each, for a total of four nodes:

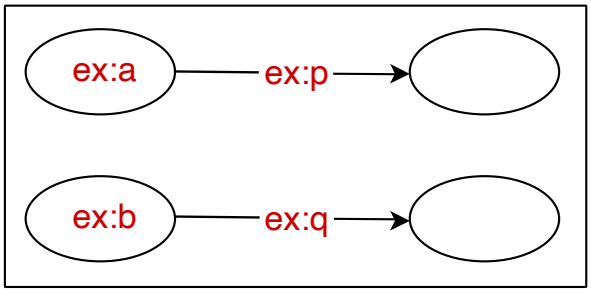
```
ex:a ex:p _:x .
```




```
ex:b ex:q _:x .
```



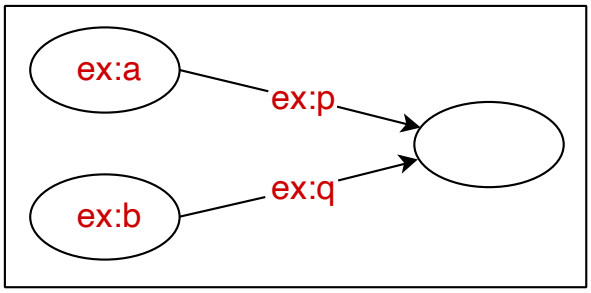
Their union would also contain four nodes:



However, if we simply concatenate these textual surface representations to form a new document, as shown below:

```
ex:a ex:p _:x .
ex:b ex:q _:x .
```

The graph for this new document contains three nodes, because the two occurrences of the blank node identifier "`_:x` now occur in a common identifier scope, and thus identify the same blank node, as shown below:



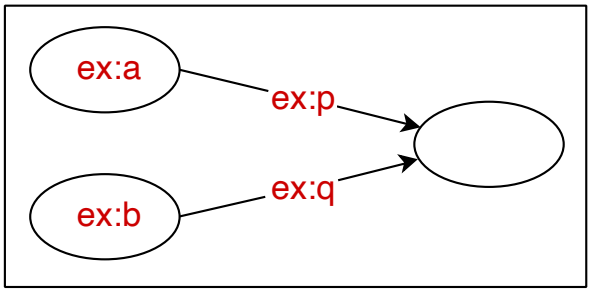
The four-node union of these two graphs is instead that shown below:

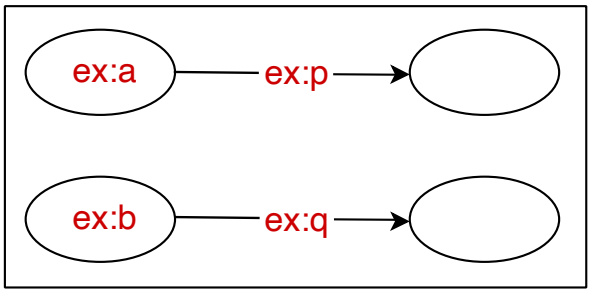
```
ex:a ex:p _:x1 .
ex:b ex:q _:x2 .
```

Here the blank node identifiers have been standardized apart to avoid conflating the distinct blank nodes. (The particular blank node identifiers used have no significance; it matters only that they are distinct.)

It is possible for two or more graphs to share a blank node, for example if they are subgraphs of a single larger graph or derived from a common source. In this case, the union of a set of graphs preserves the identity of blank nodes shared between the graphs. In general, the union of a set of RDF graphs accurately represents the same semantic content as the graphs themselves, whether or not they share blank nodes.

A related operation, called ***merging***, takes the union after forcing any shared blank nodes, which occur in more than one graph, to be distinct in each graph. The resulting graph is called the ***merge***. The merge of subgraphs of a graph may be larger than the original graph. For example, the result of merging the two singleton subgraphs of the three-node graph shown below, is the four-node graph shown beneath:





The union is always an instance of the merge. If graphs have no blank nodes in common, then their merge and union are identical.

§ 5. Simple Interpretations

This section defines the basic notions of simple interpretation and truth for RDF graphs. All semantic extensions of any vocabulary or higher-level notation encoded in RDF *MUST* conform to these minimal truth conditions. Other semantic extensions may extend and add to these, but they *MUST NOT* modify or negate them. For example, because simple interpretations are mappings which apply to IRIs, a semantic extension cannot interpret different occurrences of a single IRI differently.

The entire semantics applies to RDF graphs, not to RDF sources. An RDF source has a semantic meaning only through the graph that is its value at a given time, or in a given state. Graphs cannot change their semantics with time.

A ***simple interpretation*** I is a structure consisting of:

Definition of a simple interpretation.

1. A non-empty set IR, the resources of I, also called the domain or universe of I.

2. A set IP, the properties of I.

3. A mapping IEXT from IP into the powerset of IR x IR (the set of sets of pairs < x, y > with x and y in IR), the extension mapping for properties in I.

4. A mapping IS from IRIs into (IR union IP), the denotation mapping for symbols in I.

5. A partial mapping IL from literals into IR, the denotation mapping for literals in I.

6. An injective mapping IT from IR x IP x IR into IR, the extension mapping for triples in I.

Technical Note

Simple interpretations are required to interpret all names, and are therefore infinite. It was shown in Appendix B of RDF 1.1 Semantics spec. that RDF 1.1 could be interpreted using finite structures.

IEXT(x), called the ***extension*** of x, is a set of pairs which identify the arguments for which the property is true, that is, a binary relational extension.

The distinction between IS and IL will become significant below when the semantics of datatypes are defined. IL is allowed to be partial because some literals may fail to have a referent.

Technical Note

It is conventional to map a relation name to a relational extension directly. This however presumes that the vocabulary is segregated into relation names and individual names, and RDF makes no such assumption. Moreover, RDF allows an IRI to be used as a relation name applied to itself as an argument. Such self-application structures are used in RDFS, for example. The use of the IEXT mapping to distinguish the relation as an object from its relational extension accommodates both of these requirements. It also provides for a notion of RDFS 'class' which can be distinguished from its set-theoretic extension. A similar technique is used in the ISO/IEC Common Logic standard [*ISO24707*].

The referent of a ground RDF graph in a simple interpretation I is then given by the following rules, where the interpretation is also treated as a function from expressions (names, triples and graphs) to elements of the universe and truth values:

Semantic conditions for ground graphs.

if E is a literal then I(E) = IL(E)
if E is an IRI, then I(E) = IS(E)
if E is a ground triple term, then I(E) = IT(I(E.s), I(E.p), I(E.o)), where E.s, E.p, and E.o are the first, second, and third components of E, respectively
if E is a ground triple `s p o.`, then I(E) = true if I(p) is in IP and the pair <I(s),I(o)> is in IEXT(I(p)) otherwise I(E) = false.

if E is a ground RDF graph then I(E) = false if I(E') = false for some triple E' in E, otherwise I(E) =true.

If IL(E) is undefined for some literal E then E has no semantic value, so any triple containing it will be false, so any graph containing that triple will also be false.

The final condition implies that the empty graph (the empty set of triples) is always true.

The sets IP and IR may overlap, indeed IP can be a subset of IR. Because of the domain conditions on IEXT, the referent of the subject and object of any true triple will be in IR; so any IRI which occurs in a graph both as a predicate and as a subject or object will denote something in the intersection of IP and IR.

We observe that no IRI, not even those in the `rdf:` namespace, has any special semantic condition associated with it in a simple interpretation.

Semantic extensions may impose further constraints upon interpretation mappings by requiring some IRIs to denote in particular ways. For example, D-interpretations, described below, require some IRIs, understood as identifying and referring to datatypes, to have a fixed referent.

§ 5.1 Blank nodes

Blank nodes are treated as simply indicating the existence of a thing, without using an IRI to identify any particular thing. This is not the same as assuming that the blank node indicates an 'unknown' IRI.

Suppose I is a simple interpretation and A is a mapping from a set of blank nodes to the universe IR of I. Define the mapping [I+A] as below:

- [I+A](x)=I(x) when x is a name.
- [I+A](x)=A(x) when x is a blank node.
- [I+A](x)=IT([I+A](x.s), [I+A](x.p), [I+A](x.o)) when x is a triple term, where x.s, x.p, and x.o are the first, second, and third components of x, respectively.
- [I+A](x)=true when x is a triple; x.s, x.p, and x.o are the first, second, and third components of x, respectively; [I+A](x.p) is in IP; and the pair < [I+A](x.s), [I+A](x.o) >

is in IEXT([I+A](x.p)).

- [I+A](x)=false when x is a triple, otherwise.
- [I+A](x)=false when x is an RDF graph and [I+A](x')=false for some triple x' in x.
- [I+A](x)=true when x an RDF graph, otherwise.

Extend this mapping to triples and RDF graphs using the rules given above for ground graphs. Then the semantic conditions for an RDF graph are:

Semantic condition for blank nodes.

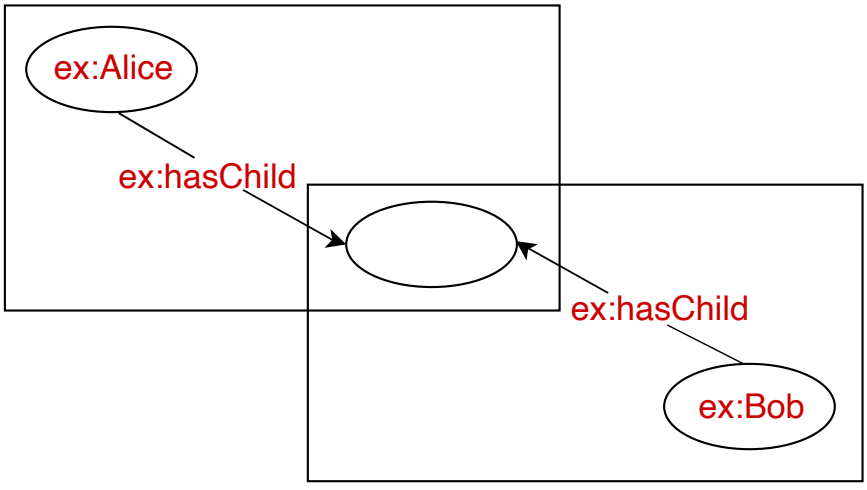
If E is an RDF graph then I(E) = true if [I+A](E) = true for some mapping A from the set of blank nodes in E to IR, otherwise I(E)= false.

Mappings from blank nodes to referents are not part of the definition of a simple interpretation, since the truth condition refers only to *some* such mapping. Blank nodes themselves differ from other nodes in not being assigned a referent by a simple interpretation, reflecting the intuition that they have no 'global' meaning.

§ 5.1.1 Shared blank nodes

This section is non-normative.

The semantics for blank nodes are stated in terms of the truth of a graph. However, when two (or more) graphs share a blank node, their meaning is not fully captured by treating them in isolation. For example, consider the overlapping graphs



and a simple interpretation I over the universe {Alice, Bob, Monica, Ruth} with: I(`ex:Alice`)=Alice, I(`ex:Bob`)=Bob, IEXT(I(`ex:hasChild`))={<Alice,Monica>, <Bob,Ruth> }

Each of the inner graphs is true under this interpretation, but the two of them together is not, because the three-node graph says that Alice and Bob have a child together. In order to capture the full meaning of graphs sharing a blank node, it is necessary to consider the union graph containing all the triples which contain the blank node.

Technical Note

RDF graphs can be viewed as conjunctions of simple atomic sentences in first-order logic, where blank nodes are free variables which are understood to be existential. Taking the union of two graphs is then analogous to syntactic conjunction in this syntax. RDF syntax has no explicit variable-binding quantifiers, so the truth conditions for any RDF graph treat the free variables in that graph as existentially quantified in that graph. Taking the union of graphs which share a blank node changes the implied quantifier scopes.

§ 5.2 Simple Entailment

Following standard terminology, we say that I (simply) ***satisfies*** E when I(E)=true, that E is (simply) ***satisfiable*** when a simple interpretation exists which satisfies it, otherwise (simply) ***unsatisfiable***, and that a graph G simply ***entails*** a graph E when every interpretation which satisfies G also satisfies E. If two graphs E and F each entail the other then they are logically ***equivalent***. If there are no (simple) interpretations that satisfy a graph then that graph is ***inconsistent***.

In later sections these notions will be adapted to other classes of interpretations, but throughout this section 'entailment' should be interpreted as meaning simple entailment.

Technical Note

We do not define a notion of entailment between sets of graphs. To determine whether a set of graphs entails a graph, the graphs in the set must first be combined into one graph, either by taking the union or the merge of the graphs. Unions preserve the common meaning of shared blank nodes, while merging effectively ignores any sharing of blank nodes. Merging the set of graphs produces the same definition of entailment by a set that was defined in the 2004 RDF 1.0 specification.

Any process which constructs a graph E from some other graph S is (simply) ***valid*** if S simply entails E in every case, otherwise ***invalid***.

The fact that an inference is valid should not be understood as meaning that any RDF application is obliged or required to make the inference. Similarly, the logical invalidity of some RDF transformation or process does not mean that the process is incorrect or prohibited. Nothing in this specification requires or prohibits any particular operations on RDF graphs or sources. Entailment and validity are concerned solely with establishing the conditions on such operations which guarantee the preservation of truth. While logically invalid processes, which do not follow valid entailments, are not prohibited, users should be aware that they may be at risk of introducing falsehoods into true RDF data. Nevertheless, particular uses of logically invalid processes may be justified and appropriate for data processing under circumstances where truth can be ensured by other means.

Entailment refers only to the truth of RDF graphs, not to their suitability for any other purpose. It is possible for an RDF graph to be fitted for a given purpose and yet validly entail another graph which is not appropriate for the same purpose. An example is the RDF test cases manifest [*RDF-TESTCASES*] which is provided as an RDF document for user convenience. This document lists examples of correct entailments by describing their antecedents and conclusions. Considered as an RDF graph, the manifest simply entails a subgraph which omits the antecedents, and would therefore be incorrect if used as a test case manifest. This is not a violation of the RDF semantic rules, but it shows that the property of "*being a correct RDF test case manifest*" is not preserved under RDF entailment, and therefore cannot be described as an RDF semantic extension. Such entailment-risky uses of RDF should be restricted to cases, as here, where it is obvious to all parties what the intended special restrictions on entailment are, in contrast with the more normal case of using RDF for the open publication of data on the Web.

§ 5.3 Properties of simple entailment and satisfiability

This section is non-normative.

The properties described here apply only to simple entailment, not to extended notions of entailment introduced in later sections. Proofs are given in B. Proofs of some results.

Every graph is simply satisfiable.

This does not always hold for extended notions of interpretation. For example, a graph containing an ill-typed literal is D-unsatisfiable.

The following ***interpolation*** **lemma**

G simply entails a graph E if and only if a subgraph of G is an instance of E.

completely characterizes simple entailment in syntactic terms. To detect whether one RDF graph simply entails another, check that there is some instance of the entailed graph which is a subset of the first graph.

Technical Note

This is clearly decidable, but it is also difficult to determine in general, since one can encode the NP-hard subgraph problem (detecting whether one mathematical graph is a subgraph of another) as detecting simple entailment between RDF graphs. This construction (due to Jeremy Carroll) uses graphs all of whose nodes are blank nodes. The complexity of checking simple entailment is reduced by having fewer blank nodes in the conclusion E. When E is a ground RDF graph, it is simply a matter of checking the subset relationship on sets of triples.

Interpolation has a number of direct consequences, for example:

The empty graph is simply entailed by any graph, and does not simply entail any graph except itself.

A graph simply entails all its subgraphs.

A graph is simply entailed by any of its instances.

If E is a lean graph and E' is a proper instance of E, then E does not simply entail E'.

If S is a subgraph of S' and S simply entails E, then S' simply entails E.

If S entails a finite graph E, then some finite subset S' of S entails E.

The property just above is called *compactness* - RDF is compact. As RDF graphs can be infinite, this is sometimes important.

> If E contains an IRI which does not occur anywhere in S, then S does not simply entail E.

The following semantic properties relate triple terms and triples asserted in a graph, and they introduce a general definition of satisfiability.

We define the ***set of propositions*** in an interpretation as follows:

> The set of propositions in an interpretation I is IPR(I) = { IT(x, y, z) | x is in IR, y is in IP, z is in IR }.

The denotation of a triple is a proposition, whether it is used as a triple term or an asserted triple. Under RDFS Interpretations (see below), a proposition is in the extension of the class `rdfs:Proposition`.

We define the ***set of facts*** in an interpretation as follows:

> The set F of facts in an interpretation I is F(I) = { IT(x, y, z) | <x, z> is in IEXT(y) }.

A fact in an interpretation is a proposition that holds in it, corresponding to a triple which is true in that interpretation.

Given a blank node mapping, we define the ***set of facts asserted by a graph*** in an interpretation as follows:

> Given a blank node mapping A, the set of all facts asserted by a graph G in an interpretation I is FEXT(G, I, A) = { IT([I+A](s), I(p), [I+A](o)) | `s p o.` is in G }.

Given a blank node mapping and an interpretation, an asserted fact in a graph is the proposition corresponding to the denotation of a triple in the graph. These asserted facts may not necessarily be among the facts in the interpretation. Intuitively, this would only be the case if the interpretation satisfies the graph.

An interpretation I (simply) satisfies a graph G if and only if there exists a blank node mapping A such that the facts asserted by the graph in the interpretation FEXT(G,I,A) are a subset of the facts of the interpretation F(I).

§ 6. Skolemization

This section is non-normative.

Skolemization is a transformation on RDF graphs which eliminates blank nodes by replacing them with "new" IRIs, which means IRIs which are coined for this purpose and are therefore guaranteed to not occur in any other RDF graph (at the time of creation). See Replacing Blank Nodes with IRIs in the RDF Concepts specification [*RDF12-CONCEPTS*] for a fuller discussion.

Suppose G is a graph containing blank nodes and sk is a skolemization mapping from the blank nodes in G to the skolem IRIs which are substituted for them, so that sk(G) is a skolemization of G. Then the semantic relationship between them can be summarized as follows.

sk(G) simply entails G (since sk(G) is an instance of G.)

G does not simply entail sk(G) (since sk(G) contains IRIs not in G.)

For any graph H, if sk(G) simply entails H then there is a graph H' such that G entails H' and H=sk(H') .

For any graph H which does not contain any of the "new" IRIs introduced into sk(G), sk(G) simply entails H if and only if G simply entails H.

The second property means that a graph is not logically equivalent to its skolemization. Nevertheless, they are in a strong sense almost interchangeable in RDF simple interpretations, as shown by the next two properties. The third property means that even when conclusions which do contain the new vocabulary are drawn with RDF simple entailment from the skolemized graph, these will exactly mirror what could have been derived with RDF simple

entailment from the original graph with the original blank nodes in place. The replacement of blank nodes with Skolem IRIs does not effectively alter what can be validly derived from the graph with RDF simple entailment, other than by giving new names to what were formerly anonymous entities. The fourth property, which is a consequence of the third, clearly shows that in some sense, a skolemization of G can "stand in for" G as far as RDF simple entailments are concerned. Using sk(G) instead of G will not affect any RDF simple entailments which do not involve the new skolem vocabulary.

§ 7. Literals and datatypes

Datatypes are identified by IRIs. Interpretations will vary according to which IRIs are recognized as denoting datatypes. We describe this using a parameter D on simple interpretations, where D is the set of ***recognized*** datatype IRIs.

The exact mechanism by which an IRI identifies a datatype is considered to be external to the semantics, but the semantics presumes that a recognized IRI identifies a unique datatype wherever it occurs. RDF processors which are not able to determine which datatype is identified by an IRI cannot recognize that IRI, and should treat any literals with that IRI as their datatype IRI as unknown names.

RDF literals and datatypes are fully described in the Datatypes section of the RDF Concepts specification [*RDF12-CONCEPTS*]. In summary: with two exceptions, RDF literals combine a string and an IRI identifying a datatype. The exceptions are language-tagged strings, which have the type `rdf:langString`, and directional language-tagged strings, which have the type `rdf:dirLangString`. Language-tagged strings have two syntactic components: a string, and a language tag; directional language-tagged strings have three syntactic components: a string, a language tag, and a base direction. A datatype is understood to define a mapping, called the ***lexical-to-value mapping***, from a lexical space (a set of strings) to values. The function ***L2V*** maps datatypes to their lexical-to-value mapping. A literal with datatype d denotes the value obtained by applying this mapping to the lexical form sss: L2V(d)(sss). If the literal string is not in the lexical space, so that the lexical-to-value mapping gives no value for the literal string, then the literal has no referent. The ***value space*** of a datatype is the range of the lexical-to-value mapping. Every literal with that type either denotes a value in the value space of the type, or fails to denote at all. An ***ill-typed*** literal is one whose datatype IRI is recognized, but

whose lexical form is not in the lexical space of the datatype identified by that IRI and thus is not in the domain of the lexical-to-value mapping of that datatype.

RDF processors are not required to recognize any datatype IRIs other than `xsd:string`, `rdf:langString`, and `rdf:dirLangString` but when IRIs listed in the Datatypes section of the RDF Concepts specification [*RDF12-CONCEPTS*] *are* recognized, they *MUST* be interpreted as described there. RDF processors *MAY* recognize other datatype IRIs, but when other datatype IRIs are recognized, the mapping between the datatype IRI and the datatype it denotes *MUST* be specified unambiguously, and *MUST* be fixed during all RDF transformations or manipulations. In practice, this can be achieved by the IRI linking to an external specification of the datatype which describes both the components of the datatype itself and the fact that the IRI identifies the datatype, thereby fixing a value of the datatype map of this IRI.

Literals with `rdf:langString` or `rdf:dirLangString` as their datatype IRI are given special treatment. The IRIs `rdf:langString` and `rdf:dirLangString` are classified as datatype IRIs and interpreted to denote a datatype, even though no L2V mapping is defined for them. The value space of `rdf:langString` is the set of all pairs of a string with a language tag. The value space of `rdf:dirLangString` is the set of all 3-tuples of a string, a language tag, and a base direction. The semantics of literals with either of these as their datatype are given below.

RDF allows any IRI to be used in a literal, even when it is not recognized as referring to a datatype. Literals with such an "unknown" datatype IRI, which is not in the set of recognized datatypes, *SHOULD NOT* be treated as errors, although RDF applications *MAY* issue a warning. Such literals *SHOULD* be treated like IRIs and assumed to denote some thing in the universe IR. RDF processors which do not recognize a datatype IRI will not be able to detect some entailments which are visible to one which does. For example, the fact that

```
ex:a ex:p "20.0000"^^xsd:decimal .
```

entails

```
ex:a ex:p "20.0"^^xsd:decimal .
```

will not be visible to a processor which does not recognize the datatype IRI `xsd:decimal`.

§ 7.1 D-interpretations

Let D be a set of IRIs identifying datatypes. A **(simple)** ***D-interpretation*** is a simple interpretation which satisfies the following conditions:

Semantic conditions for literals.

If `rdf:langString` is in D, then for every language-tagged string E with lexical form sss and language tag ttt, IL(E)= < sss, ttt' >, where ttt' is ttt converted to lower case using US-ASCII rules
If `rdf:dirLangString` is in D, then for every directional language-tagged string E with lexical form sss, language tag ttt, and base direction bbb, IL(E)= < sss, ttt', bbb >, where ttt' is ttt converted to lower case using US-ASCII rules
For every other IRI aaa in D, I(aaa) is the datatype identified by aaa, and for every literal "sss"^^aaa, IL("sss"^^aaa) = L2V(I(aaa))(sss)

If the literal is ill-typed then the L2V(I(aaa)) mapping has no value, and so the literal cannot denote anything. In this case, any triple containing the literal must be false. Thus, any triple, and hence any graph, containing an ill-typed literal will be D-unsatisfiable, i.e. false in every D-interpretation. This applies only to literals typed with recognized datatype IRIs in D; literals with an unrecognized type IRI are not ill-typed and cannot give rise to a D-unsatisfiable graph.

The special datatypes `rdf:langString` and `rdf:dirLangString` have no ill-typed literals. Any syntactically legal literal with one of these types will denote a value in every D-interpretation where D includes `rdf:langString` or `rdf:dirLangString`. The only ill-typed literals of type `xsd:string` are those containing a Unicode code point which does not match the *Char* production in *Extensible Markup Language (XML) 1.1 (Second Edition)* [*XML11*]. Such strings cannot be written in an XML-compatible surface syntax.

§ 7.2 Datatype entailment

A graph is (simply) ***D-satisfiable*** or ***satisfiable recognizing D*** when it has the value true in some D-interpretation, and a graph S (simply) ***D-entails*** or ***entails recognizing D*** a graph G when every D-interpretation which satisfies S also D-satisfies G.

Unlike the case with simple interpretations, it is possible for a graph to have no satisfying D-interpretations i.e. to be ***D-unsatisfiable***. RDF processors *MAY* treat an unsatisfiable graph as signaling an error condition, but this is not required.

A D-unsatisfiable graph D-entails any graph.

Technical Note

The fact that an unsatisfiable statement entails any other statement has been known since antiquity. It is called the principle of *ex falso quodlibet*. It should not be interpreted to mean that it is necessary, or even permissible, to actually draw any conclusion from an unsatisfiable graph.

In all of this language, 'D' is being used as a parameter to represent some set of datatype IRIs, and different D sets will yield different notions of satisfiability and entailment. The more datatypes are recognized, the stronger is the entailment, so that if D ⊂ E and S E-entails G then S must D-entail G. Simple entailment is { }-entailment, i.e. D-entailment when D is the empty set, so if S D-entails G then S simply entails G.

§ 7.2.1 Patterns of datatype entailment

This section is non-normative.

Unlike simple entailment, it is not possible to give a single syntactic criterion to detect all D-entailments, which can hold because of particular properties of the lexical-to-value mappings of the recognized datatypes. For example, if D contains `xsd:decimal` then

```
ex:a ex:p "25.0"^^xsd:decimal .
```

D-entails

```
ex:a ex:p "25"^^xsd:decimal .
```

In general, any triple containing a literal with a recognized datatype IRI D-entails another literal when the lexical strings of the literals map to the same value under the lexical-to-value

map of the datatype. If two different datatypes in D map lexical strings to a common value, then a triple containing a literal typed with one datatype may D-entail another triple containing a literal typed with a different datatype. For example, `"25"^^xsd:integer` and `"25.0"^^xsd:decimal` have the same value, so the above also D-entails

```
ex:a ex:p "25"^^xsd:integer .
```

when D also contains `xsd:integer`.

(There is a W3C Note [*SWBP-XSCH-DATATYPES*] containing a long discussion of literal values.)

Ill-typed literals are the only way in which a graph can be simply D-unsatisfiable, but datatypes can give rise to a variety of other unsatisfiable graphs when combined with the RDFS vocabulary, defined later.

§ 8. RDF Interpretations

RDF interpretations impose extra semantic conditions on `xsd:string` and part of the infinite set of IRIs with the namespace prefix `rdf:` .

RDF vocabulary
`rdf:type rdf:reifies rdf:subject rdf:predicate rdf:object rdf:first rdf:rest rdf:value rdf:nil rdf:List rdf:langString rdf:dirLangString rdf:Property rdf:_1 rdf:_2 ...`

An ***RDF interpretation*** **recognizing D** is a D-interpretation I where D includes `rdf:langString`, `rdf:dirLangString`, and `xsd:string`, and which satisfies:

RDF semantic conditions.

x is in IP if and only if <x, I(`rdf:Property`)> is in IEXT(I(`rdf:type`))
For every IRI aaa in D, < x, I(aaa) > is in IEXT(I(`rdf:type`)) if and only if x is in the value space of I(aaa)

and satisfies every triple in the following infinite set:

RDF axioms.

```
rdf:type rdf:type rdf:Property .
rdf:subject rdf:type rdf:Property .
rdf:predicate rdf:type rdf:Property .
rdf:object rdf:type rdf:Property .
rdf:reifies rdf:type rdf:Property .
rdf:first rdf:type rdf:Property .
rdf:rest rdf:type rdf:Property .
rdf:value rdf:type rdf:Property .
rdf:nil rdf:type rdf:List .
rdf:_1 rdf:type rdf:Property .
rdf:_2 rdf:type rdf:Property .
...
```

RDF intrepretations impose no particular normative semantics on the rest of the RDF vocabulary.

The datatype IRIs `rdf:langString`, `rdf:dirLangString`, and `xsd:string` *MUST* be recognized by all RDF interpretations.

Three other datatypes — `rdf:XMLLiteral`, `rdf:HTML`, and `rdf:JSON` — are defined in the RDF Concepts specification [*RDF12-CONCEPTS*]. RDF-D interpretations *MAY* fail to recognize these datatypes.

§ 8.1 RDF entailment

S ***RDF entails*** E **recognizing D** when every RDF interpretation recognizing D which satisfies S also satisfies E. When D is {`rdf:langString`, `rdf:dirLangString`, `xsd:string`} then we simply say S **RDF entails** E. E is ***RDF unsatisfiable*** **(recognizing D)** when it has no satisfying RDF interpretation (recognizing D).

The properties of simple entailment described earlier do not all apply to RDF entailment. For example, all the RDF axioms are true in every RDF interpretation, and so are RDF entailed by the empty graph, contradicting interpolation for RDF entailment.

§ 8.1.1 Patterns of RDF entailment

This section is non-normative.

The last semantic condition in the above table gives the following entailment pattern for recognized datatype IRIs:

RDF entailment pattern.

	if S contains	**then S RDF entails, recognizing D**
rdfD1	Any triple ttt such that "sss"^^ddd appears in ttt for ddd in D	ttt ["sss"^^ddd/_:nnn] _:nnn `rdf:type` ddd .

Note, this is valid even when the literal is ill-typed, since an unsatisfiable graph entails any triple.

For example,

```
ex:a ex:p "123"^^xsd:integer .
```

RDF entails recognizing {`xsd:integer`}

```
ex:a ex:p _:x .
_:x rdf:type xsd:integer .
```

The last semantic condition above also justifies the following entailment pattern:

	any S	**then S RDF entails, recognizing D**
rdfD1a	for ddd in D with non-empty value space	_:nnn `rdf:type` ddd .

In addition, the first RDF semantic condition justifies the following entailment pattern:

	if the triple appears in S	**then S RDF entails, recognizing D**
rdfD2	xxx aaa yyy .	aaa `rdf:type rdf:Property` .

So that the above example also RDF entails

```
ex:p rdf:type rdf:Property .
```

recognizing {`xsd:integer`}.

Some datatypes support idiosyncratic entailment patterns which do not hold for other datatypes. For example,

```
ex:a ex:p "true"^^xsd:boolean .
ex:a ex:p "false"^^xsd:boolean .
ex:v rdf:type xsd:boolean .
```

together RDF entail

```
ex:a ex:p ex:v .
```

recognizing {`xsd:boolean`}.

In addition, the semantic conditions on value spaces may produce other unsatisfiable graphs. For example, when D contains `xsd:integer` and `xsd:boolean`, then the following is RDF unsatisfiable recognizing D:

```
_:x rdf:type xsd:boolean .
_:x rdf:type xsd:integer .
```

§ 9. RDFS Interpretations

RDF Schema [*RDF12-SCHEMA*] extends RDF to a larger vocabulary with more complex semantic constraints:

RDFS vocabulary

`rdfs:domain rdfs:range rdfs:Resource rdfs:Literal rdfs:Datatype`
`rdfs:Class rdfs:subClassOf rdfs:subPropertyOf rdfs:Proposition`
`rdfs:member rdfs:Container rdfs:ContainerMembershipProperty`
`rdfs:comment rdfs:seeAlso rdfs:isDefinedBy rdfs:label`

(rdfs:comment, rdfs:seeAlso, rdfs:isDefinedBy and rdfs:label are included here because some constraints which apply to their use can be stated using rdfs:domain, rdfs:range and rdfs:subPropertyOf. Other than this, the formal semantics does not constrain their meanings.)

It is convenient to state the RDFS semantics in terms of a new semantic construct, a ***class***, i.e. a resource which represents a set of things in the universe which all have that class as a value of their rdf:type property. Classes are defined to be things of type rdfs:Class, and the set of all classes in an interpretation will be called IC. The semantic conditions are stated in terms of a mapping ICEXT (for the *C*lass *Ext*ension in I) from IC to the set of subsets of IR.

A class may have an empty class extension. Two different classes can have the same class extension. The class extension of rdfs:Class contains the class rdfs:Class.

RDFS also introduces the class rdfs:Proposition, whose extension is exactly the set of propositions as defined in 5.3 Properties of simple entailment and satisfiability. This class is also declared as rdfs:range of the rdf:reifies property. In other words, the object of a reifying triple always denotes a proposition.

An ***RDFS interpretation*** (**recognizing D**) is an RDF interpretation (recognizing D) I which satisfies the semantic conditions in the following table, and all the triples in the subsequent table of RDFS axiomatic triples.

RDFS semantic conditions.

ICEXT(y) is defined to be { x : < x,y > is in IEXT(I(rdf:type)) }

IC is defined to be ICEXT(I(rdfs:Class))

LV is defined to be ICEXT(I(rdfs:Literal))

ICEXT(I(rdfs:Resource)) = IR

ICEXT(I(rdf:langString)) is the set {I(E) : E a language-tagged string }

ICEXT(I(rdf:dirLangString)) is the set {I(E) : E a directional language-tagged string }

for every other IRI aaa in D, ICEXT(I(aaa)) is the value space of I(aaa)

for every IRI aaa in D, I(aaa) is in ICEXT(I(rdfs:Datatype))

If < x,y > is in IEXT(I(rdfs:domain)) and < u,v > is in IEXT(x) then u is in ICEXT(y)
If < x,y > is in IEXT(I(rdfs:range)) and < u,v > is in IEXT(x) then v is in ICEXT(y)
IEXT(I(rdfs:subPropertyOf)) is transitive and reflexive on IP
If <x,y> is in IEXT(I(rdfs:subPropertyOf)) then x and y are in IP and IEXT(x) is a subset of IEXT(y)
If x is in IC then < x, I(rdfs:Resource) > is in IEXT(I(rdfs:subClassOf))
IEXT(I(rdfs:subClassOf)) is transitive and reflexive on IC
If < x,y > is in IEXT(I(rdfs:subClassOf)) then x and y are in IC and ICEXT(x) is a subset of ICEXT(y)
If x is in ICEXT(I(rdfs:ContainerMembershipProperty)) then: < x, I(rdfs:member) > is in IEXT(I(rdfs:subPropertyOf))
If x is in ICEXT(I(rdfs:Datatype)) then < x, I(rdfs:Literal) > is in IEXT(I(rdfs:subClassOf))
If exist x,y,z such that IT(x,z,y)=r then < r,I(rdfs:Proposition)> is in IEXT(I(rdf:type))

RDFS axiomatic triples.

```
rdf:type rdfs:domain rdfs:Resource .
rdf:reifies rdfs:domain rdfs:Resource .
rdfs:domain rdfs:domain rdf:Property .
rdfs:range rdfs:domain rdf:Property .
rdfs:subPropertyOf rdfs:domain rdf:Property .
rdfs:subClassOf rdfs:domain rdfs:Class .
rdf:subject rdfs:domain rdf:Statement .
rdf:predicate rdfs:domain rdf:Statement .
rdf:object rdfs:domain rdf:Statement .
rdfs:member rdfs:domain rdfs:Resource .
rdf:first rdfs:domain rdf:List .
rdf:rest rdfs:domain rdf:List .
rdfs:seeAlso rdfs:domain rdfs:Resource .
rdfs:isDefinedBy rdfs:domain rdfs:Resource .
rdfs:comment rdfs:domain rdfs:Resource .
rdfs:label rdfs:domain rdfs:Resource .
rdf:value rdfs:domain rdfs:Resource .

rdf:type rdfs:range rdfs:Class .
rdf:reifies rdfs:range rdfs:Proposition .
rdfs:domain rdfs:range rdfs:Class .
rdfs:range rdfs:range rdfs:Class .
rdfs:subPropertyOf rdfs:range rdf:Property .
rdfs:subClassOf rdfs:range rdfs:Class .
rdf:subject rdfs:range rdfs:Resource .
rdf:predicate rdfs:range rdfs:Resource .
rdf:object rdfs:range rdfs:Resource .
rdfs:member rdfs:range rdfs:Resource .
rdf:first rdfs:range rdfs:Resource .
rdf:rest rdfs:range rdf:List .
rdfs:seeAlso rdfs:range rdfs:Resource .
rdfs:isDefinedBy rdfs:range rdfs:Resource .
rdfs:comment rdfs:range rdfs:Literal .
rdfs:label rdfs:range rdfs:Literal .
rdf:value rdfs:range rdfs:Resource .
```

```
rdf:Alt rdfs:subClassOf rdfs:Container .
rdf:Bag rdfs:subClassOf rdfs:Container .
rdf:Seq rdfs:subClassOf rdfs:Container .
rdfs:ContainerMembershipProperty rdfs:subClassOf rdf:Property .

rdfs:isDefinedBy rdfs:subPropertyOf rdfs:seeAlso .

rdfs:Datatype rdfs:subClassOf rdfs:Class .

rdf:_1 rdf:type rdfs:ContainerMembershipProperty .
rdf:_1 rdfs:domain rdfs:Resource .
rdf:_1 rdfs:range rdfs:Resource .
rdf:_2 rdf:type rdfs:ContainerMembershipProperty .
rdf:_2 rdfs:domain rdfs:Resource .
rdf:_2 rdfs:range rdfs:Resource .
...
```

Since I is an RDF interpretation, the first condition implies that IP = ICEXT(I(`rdf:Property`)).

The semantic conditions on RDF interpretations, together with the RDFS conditions on ICEXT, mean that every recognized datatype can be treated as a class whose extension is the value space of the datatype, and every literal with that datatype either fails to denote, or denotes a value in that class.

When using RDFS semantics, the referents of all recognized datatype IRIs can be considered to be in the class `rdfs:Datatype`.

The axioms and conditions listed above have some redundancy. For example, all but one of the RDF axiomatic triples can be derived from the RDFS axiomatic triples and the semantic conditions on ICEXT, `rdfs:domain` and `rdfs:range`.

Other triples which must be true in all RDFS interpretations include the following. This is not a complete set.

Some rdfs-valid triples.

```
rdfs:Resource rdf:type rdfs:Class .
rdfs:Class rdf:type rdfs:Class .
```

```
rdfs:Literal rdf:type rdfs:Class .
rdf:XMLLiteral rdf:type rdfs:Class .
rdf:HTML rdf:type rdfs:Class .
rdfs:Datatype rdf:type rdfs:Class .
rdf:Seq rdf:type rdfs:Class .
rdf:Bag rdf:type rdfs:Class .
rdf:Alt rdf:type rdfs:Class .
rdfs:Container rdf:type rdfs:Class .
rdf:List rdf:type rdfs:Class .
rdfs:ContainerMembershipProperty rdf:type rdfs:Class .
rdf:Property rdf:type rdfs:Class .
rdf:Statement rdf:type rdfs:Class .

rdfs:domain rdf:type rdf:Property .
rdfs:range rdf:type rdf:Property .
rdfs:subPropertyOf rdf:type rdf:Property .
rdfs:subClassOf rdf:type rdf:Property .
rdfs:member rdf:type rdf:Property .
rdfs:seeAlso rdf:type rdf:Property .
rdfs:isDefinedBy rdf:type rdf:Property .
rdfs:comment rdf:type rdf:Property .
rdfs:label rdf:type rdf:Property .
```

RDFS does not partition the universe into disjoint categories of classes, properties and individuals. Anything in the universe can be used as a class or as a property, or both, while retaining its status as an individual which may be in classes and have properties. Thus, RDFS permits classes which contain other classes, classes of properties, properties of classes, etc. As the axiomatic triples above illustrate, it also permits classes which contain themselves and properties which apply to themselves. A property of a class is not necessarily a property of its members, nor vice versa.

§ 9.1 A note on rdfs:Literal

This section is non-normative.

The class `rdfs:Literal` is not the class of literals, but rather that of literal values, which may also be denoted by IRIs. For example, LV does not contain the literal `"foodle"^^xsd:string` but it does contain the string "foodle".

A triple of the form

```
ex:a rdf:type rdfs:Literal .
```

is consistent even though its subject is an IRI rather than a literal. It says that the IRI '`ex:a`' denotes a literal value, which is quite possible since literal values are things in the universe. Blank nodes may range over literal values, for the same reason.

§ 9.2 RDFS entailment

S ***RDFS entails*** E **recognizing D** when every RDFS interpretation recognizing D which satisfies S also satisfies E.

Since every RDFS interpretation is an RDF interpretation, if S RDFS entails E then S also RDF entails E; but RDFS entailment is stronger than RDF entailment. Even the empty graph has a large number of RDFS entailments which are not RDF entailments, for example all triples of the form

aaa `rdf:type rdfs:Resource .`

where aaa is an IRI, are true in all RDFS interpretations.

§ 9.2.1 Patterns of RDFS entailment

This section is non-normative.

RDFS entailment holds for all the following patterns, which correspond closely to the RDFS semantic conditions:

RDFS entailment patterns.

		then S RDFS entails recognizing

	If S contains:	D:
rdfs1	any IRI aaa in D	aaa `rdf:type rdfs:Datatype` .
rdfs2	aaa `rdfs:domain` xxx . yyy aaa zzz .	yyy `rdf:type` xxx .
rdfs3	aaa `rdfs:range` xxx . yyy aaa zzz .	zzz `rdf:type` xxx .
rdfs4	Any triple ttt such that xxx appears in ttt	xxx `rdf:type rdfs:Resource` .
rdfs5	xxx `rdfs:subPropertyOf` yyy . yyy `rdfs:subPropertyOf` zzz .	xxx `rdfs:subPropertyOf` zzz .
rdfs6	xxx `rdf:type rdf:Property` .	xxx `rdfs:subPropertyOf` xxx .
rdfs7	aaa `rdfs:subPropertyOf` bbb . xxx aaa yyy .	xxx bbb yyy .
rdfs8	xxx `rdf:type rdfs:Class` .	xxx `rdfs:subClassOf rdfs:Resource` .
rdfs9	xxx `rdfs:subClassOf` yyy . zzz `rdf:type` xxx .	zzz `rdf:type` yyy .
rdfs10	xxx `rdf:type rdfs:Class` .	xxx `rdfs:subClassOf` xxx .
rdfs11	xxx `rdfs:subClassOf` yyy . yyy `rdfs:subClassOf` zzz .	xxx `rdfs:subClassOf` zzz .
rdfs12	xxx `rdf:type rdfs:ContainerMembershipProperty` .	xxx `rdfs:subPropertyOf rdfs:member` .
rdfs13	xxx `rdf:type rdfs:Datatype` .	xxx `rdfs:subClassOf rdfs:Literal` .

rdfs14	Any triple ttt such that <<(aaa bbb ccc)>> appears in ttt	ttt [<<(aaa bbb ccc)>>/_:nnn] _:nnn `rdf:type rdfs:Proposition .`

As an example of a RDFS entailment involving triple terms using the entailment pattern rdfs14, the graph below RDFS entails the triples that follow:

```
ex:a ex:b <<( ex:c ex:d <<(ex:e ex:f ex:g)>> )>> .
```

```
ex:a ex:b <<( ex:c ex:d _:b1 )>> .
ex:a ex:b _:b2 .
_:b1 rdf:type rdfs:Proposition .
_:b2 rdf:type rdfs:Proposition .
```

RDFS provides several new ways to be unsatisfiable recognizing D. For example, the following graph is RDFS unsatisfiable recognizing {`xsd:integer`, `xsd:boolean`}:

```
ex:p rdfs:domain xsd:boolean .
ex:a rdf:type xsd:integer .
ex:a ex:p ex:c .
```

§ 10. RDF Datasets

RDF datasets, defined in RDF Concepts [*RDF12-CONCEPTS*], package up zero or more named RDF graphs along with a single unnamed, default RDF graph. The graphs in a single dataset may share blank nodes. SPARQL [*SPARQL12-QUERY*] associates graph names with graphs to allow queries to be directed against particular graphs.

Graph names in a dataset may denote something other than the graph they are paired with. This allows IRIs or blank nodes denoting other kinds of entities, such as persons, to be used in a dataset to identify graphs of information relevant to the entity denoted by the graph name.

When a graph name is used inside RDF triples in a dataset it may or may not denote the graph it names. The semantics does not require, nor should RDF engines presume, without some external reason to do so, that graph names used in RDF triples denote the graph they name.

RDF datasets *MAY* be used to express RDF content. When used in this way, a dataset *SHOULD* be understood to have at least the same content as its default graph. Note however that replacing the default graph of a dataset by a logically equivalent graph will not in general produce a structurally similar dataset, since it may for example disrupt co-occurrences of blank nodes between the default graph and other graphs in the dataset, which may be important for reasons other than the semantics of the graphs in the dataset.

Other semantic extensions and entailment regimes *MAY* place further semantic conditions and restrictions on RDF datasets, just as with RDF graphs. One such extension, for example, could set up a modal-like interpretation structure so that entailment between datasets would require RDF graph entailments between the graphs with the same name (adding in empty graphs as required).

Definition of RDF dataset merge

Given two or more RDF datasets, their ***RDF dataset merge*** is defined by the following procedure:

1. Force any blank nodes shared between the datasets to be unique in each dataset.
2. Label each triple in the datasets (a) with the name (a URI or a blank node) of the graph that includes it, and (b) with a special label "default" if it is included in a default graph.
3. Let the default graph of the RDF dataset merge be the set of triples with the label "default".
4. For each label except the label "default", let the name of a named graph in the RDF dataset merge be that label, and let that graph be the set of triples with that label.

§ 11. Appendices

§ A. Entailment rules

This section is non-normative.

Note: This section is carried over from RDF 1.1 and is included here to show how sound and complete inference rules might be constructed for the current versions of RDF and RDFS. It is believed that at most minor changes to the entailment rules here will be needed for sound and complete RDF and RDFS entailment.

(*This section is based on work described more fully in two papers by ter Horst,* [*HORST04*] *and* [*HORST05*]*, which should be consulted for technical details and proofs.*)

The RDF and RDFS entailment patterns listed in the above tables can be viewed as left-to-right rules which add the entailed conclusion to a graph. These rule sets can be used to check RDF (or RDFS) entailment between graphs S and E, by the following sequence of operations:

1. Add to S all the RDF (or RDF and RDFS) axiomatic triples except those containing the container membership property IRIs `rdf:_1, rdf:_2, ...`
2. For every container membership property IRI which occurs in E, add the RDF (or RDF and RDFS) axiomatic triples which contain that IRI.
3. For every IRI `aaa` used in E, add `aaa rdf:type rdfs:Resource` to S.
4. Apply the RDF (or RDF and RDFS) inference patterns as rules, adding each conclusion to the graph, to exhaustion; that is, until they generate no new triples.
5. Determine if E has an instance which is a subset of the set, i.e., whether the enlarged set simply entails E.

This process is clearly correct, in that if it gives a positive result then indeed S does RDF (RDFS) entail E. It is not, however, complete: there are cases of S entailing E which are not detectable by this process. Examples include:

	RDF entails

`ex:a ex:p "string"^^xsd:string .`
`ex:b ex:q "string"^^xsd:string .`
	`ex:a ex:p _:b .`
	`ex:b ex:q _:b .`
	`_:b rdf:type xsd:string .`

	RDFS entails
`ex:a rdfs:subPropertyOf _:b .`	`ex:d rdf:type ex:c .`

`rdfs:domain ex:c .` `ex:d ex:a ex:e .`	
	RDFS entails
`ex:a ex:b ex:c .`	`<<(ex:a ex:b ex:c)>> rdf:type` `rdfs:Proposition .`

These examples can be handled by allowing the rules to apply to a generalization of the RDF syntax in which literals and triple terms may occur in subject position and blank nodes may occur in predicate position.

Consider generalized RDF triples, graphs, and datasets instead of RDF triples, graphs and datasets (extending the generalization used by Horst [*HORST04*] and following exactly the terms used in OWL Profiles [*OWL2-PROFILES*]). The semantics described in this document applies to the generalization without change, so that the notions of interpretation, satisfiability and entailment can be used freely. Then we can replace the first RDF entailment pattern with the simpler and more direct

G-RDF-D entailment pattern.

	if S contains	**then S RDF entails, recognizing D**
GrdfD1	Any triple ttt such that "sss"^^ddd appears in ttt for ddd in D	"sss"^^ddd `rdf:type` ddd `.`

which gives the entailments;

```
ex:a ex:p "string"^^xsd:string .
ex:b ex:q "string"^^xsd:string .
"string"^^xsd:string rdf:type xsd:string .
```
by GrdfD1

which is an instance (in generalized RDF) of the desired conclusion, above.

The second example can be derived using the RDFS rules:

```
ex:a rdfs:subPropertyOf _:b .
_:b rdfs:domain ex:c .
ex:d ex:a ex:e .
```

`ex:d _:b ex:e .` by rdfs7
`ex:d rdf:type ex:c .` by rdfs2

Where the entailment patterns have been applied to generalized RDF syntax but yield a final conclusion which is legal RDF.

The entailment pattern for generalized RDF with symmetric RDF triples, considering that, according to the semantics, the denotation of triple terms should be of type `rdfs:Proposition`, is the following:

RDFS-T entailment pattern.

	if S contains	**then S RDFS entails**
Grdfs14	`yyy rdf:type rdf:Property .` `xxx rdf:type rdfs:Resource . zzz rdf:type rdfs:Resource .`	`<<(xxx yyy zzz)>> rdf:type rdfs:Proposition .`

With the generalized syntax, these rules are postulated to be complete for both RDF and RDFS entailment. Stated exactly:

Let S and E be RDF graphs. Define the ***generalized RDF (RDFS) closure*** **of S towards E** to be the set obtained by the following procedure.

1. Add to S all the RDF (and RDFS) axiomatic triples which do not contain any container membership property IRI.
2. For each container membership property IRI which occurs in E, add the RDF (and RDFS) axiomatic triples which contain that IRI.
3. If no triples were added in step 2, add the RDF (and RDFS) axiomatic triples which contain `rdf:_1`.
4. Apply the rule GrdfD1 (and rdfs1 and rdfs4) but using E instead of S in the antecedent.
5. Apply rule Grdfs14.
6. Apply the rules GrdfD1, rdfD1a, and rdfD2 (and the rules rdfs1 through rdfs13), with D= {`rdf:langString`, `rdf:dirLangString`, `xsd:string`}, to the set in all possible ways, to exhaustion.

If these rules are complete, they would give rise to the following completeness result:

If S is RDF consistent (RDFS consistent), then S RDF entails (RDFS entails) E just when the generalized RDF (RDFS) closure of S towards E simply entails E.

The closures are finite. The generation process is decidable and of polynomial complexity. Detecting simple entailment is NP-complete in general, but of low polynomial order when E contains no blank nodes.

Every RDF(S) closure, even starting with the empty graph, will contain all RDF(S) tautologies which can be expressed using the vocabulary of the entailing and entailed graphs plus the RDF and RDFS vocabularies. In practice there is little utility in re-deriving these and a subset of the rules can be used to establish most entailments of practical interest.

If it is important to stay within legal RDF syntax, rule rdfD1 may be used instead of GrdfD1, and the introduced blank node can be used as a substitute for the literal in subsequent derivations. The resulting set of rules will not however be complete.

As noted earlier, detecting datatype entailment for larger sets of datatype IRIs requires attention to idiosyncratic properties of the particular datatypes.

§ B. Proofs of some results

This section is non-normative.

The empty graph is simply entailed by any graph, and does not simply entail any graph except itself.

The empty graph is true in all simple interpretations, so is entailed by any graph. If G contains a triple <a b c>, then any simple interpretation I with IEXT(I(b))={ } makes G false; so the empty graph does not entail G. QED.

A graph simply entails all its subgraphs.

If I satisfies G then it satisfies every triple in G, hence every triple in any subset of G. QED.

A graph is simply entailed by any of its instances.

Suppose H is an instance of G with the instantiation mapping M, and that I satisfies H. For blank nodes n in G which are not in H define A(n)=I(M(n)); then I+A satisfies G, so I satisfies G. QED.

Every graph is simply satisfiable.

Consider the simple interpretation with universe {x} union all (abstract) triple terms which can be formed from x, IEXT(x)= <x,x>, IS(aaa)=x, IL(aaa) = x, IT(yyy,x,zzz) = <yyy,x,zzz>, for any term aaa and any elements of the domain of discourse yyy and zzz. This interpretation satisfies every RDF graph. QED.

G simply entails a graph E if and only if a subgraph of G is an instance of E.

If a subgraph E' of G is an instance of E then G entails E' which entails E, so G entails E. Now suppose G entails E, and consider the Herbrand interpretation I of G defined as follows. IR contains the names, triple terms, and blank nodes which appear in the graph, with I(n)=n for each name n and IT(s,p,o)=<s,p,o>; n is in IP and <a, b> is in IEXT(n), only when the triple <a n b> is in the graph. (For IRIs which do not occur in the graph, assign them values in IR at random.) I satisfies every triple <s p o> in E; that is, for some mapping A from the blank nodes of E to the vocabulary of G, the triple <[I+A](s) I(p) [I+A](o)> occurs in G. But this is an instance of <s p o> under the instance mapping A; so an instance of E is a subgraph of G. QED.

if E is lean and E' is a proper instance of E, then E does not simply entail E'.

Suppose E entails E', then a subgraph of E is an instance of E', which is a proper instance of E; so a subgraph of E is a proper instance of E, so E is not lean. QED.

If E contains an IRI which does not occur in S, then S does not simply entail E.

IF S entails E then a subgraph of S is an instance of E, so every IRI in E must occur in that subgraph, so must occur in S. QED.

For any graph H, if sk(G) simply entails H then there is a graph H' such that G entails H' and

H=sk(H').

The skolemization mapping sk substitutes a unique new IRI for each blank node, so it is 1:1, so has an inverse. Define ks to be the inverse mapping which replaces each skolem IRI by the blank node it replaced. Since sk(G) entails H, a subgraph of sk(G) is an instance of H, say A(H) for some instance mapping A on the blank nodes in H. Then ks(A(H)) is a subgraph of G; and ks(A(H))=A(ks(H)) since the domains of A and ks are disjoint. So ks(H) has an instance which is a subgraph of G, so is entailed by G; and H=sk(ks(H)). QED.

For any graph H which does not contain any of the "new" IRIs introduced into sk(G), sk(G) simply entails H if and only if G simply entails H.

Using the terminology in the previous proof: if H does not contain any skolem IRIs, then H=ks(H). So if sk(G) entails H then G entails ks(H)=H; and if G entails H then sk(G) entails G entails H, so sk(G) entails H. QED.

§ C. Semantic Adequacy Considerations

This section is non-normative.

The semantic condition on the triple term mapping IT induces potentially unwanted *non-well founded models* for propositions.

For example, the following asserted triple
:s :p <<(:a :b :c)>> .
is satisfied, among many interpretations, in an interpretation I in which IT maps the denotation of the triple term <<(:a :b :c)>> to the denotation of its object — i.e., IT(I(:a), I(:b), I(:c)) = I(:c). Observe that I(:c) is a *proposition* in the interpretation I, namely it is an element of the set IPR(I) — as defined in Section 5.3 Properties of simple entailment and satisfiability — and therefore I(:c) is an instance of the denotation of `rdfs:Proposition`.

In this interpretation, there is a proposition about a relation involving the proposition itself — for example, it could be intended as the proposition “the barber denies this very proposition” — leading to a self-referential paradox.

However, it should be noted that:

a. Interpretations like the above do not induce any formal paradox, since the notion of *proposition* in RDF is very weak.

b. Most importantly, it can be shown that even if we strengthen the semantic condition on IT in order to forbid paradoxical self-referential interpretations like the above — e.g., by requiring the mapping IT to be well-founded — neither the simple, RDF, nor RDFS entailments change.

Therefore, we do not enforce a well-foundedness condition on the definition of IT, since this would have no practical consequence for simple, RDF, or RDFS entailments. However, such a well-foundedness condition may be necessary in extensions of RDF in which the indifference of entailment with respect to non-well-founded interpretations does not hold anymore, such as in the case of RDF extended with `owl:sameAs`.

§ D. Privacy Considerations

This section is non-normative.

See Privacy Considerations in *RDF 1.2 Concepts and Abstract Data Model* [*RDF12-CONCEPTS*].

§ E. Security Considerations

This section is non-normative.

See Security Considerations in *RDF 1.2 Concepts and Abstract Data Model* [*RDF12-CONCEPTS*].

§ F. Acknowledgments

This section is non-normative.

In addition to the editors, the following people have contributed to the RDF 1.2 version: Andy Seaborne, Antoine Zimmermann, Denis Ah-Kang, doerthe, Dominik Tomaszuk, Gregg Kellogg, Peter F. Patel-Schneider, Pierre-Antoine Champin, Ruben Taelman, and Ted Thibodeau Jr

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

The basic idea of using an explicit extension mapping to allow self-application without violating the axiom of foundation was suggested by Christopher Menzel. The generalized RDF syntax used in A. Entailment rules, and the example showing the need for it, were suggested by Herman ter Horst, who also proved completeness and complexity results for the rule sets of RDF 1.1. Jeremy Carroll first showed that simple entailment is NP-complete in general. Antoine Zimmerman suggested several simplifications and improvements to the proofs and presentation.

The RDF 1.1 editors acknowledge valuable contributions from Thomas Baker, Dan Brickley, Gavin Carothers, Jeremy Carroll, Pierre-Antoine Champin, Richard Cyganiak, Martin J. Dürst, Alex Hall, Steve Harris, Ivan Herman, Eric Prud'hommeaux, Andy Seaborne, David Wood and Antoine Zimmermann.

This specification draws upon the earlier specification for RDF semantics [*RDF-MT*], whose editor acknowledged valuable inputs from Jeremy Carroll, Dan Connolly, Jan Grant, R. V. Guha, Herman ter Horst, Graham Klyne, Ora Lassila, Brian McBride, Sergey Melnick, Peter Patel-Schneider, Jos De Roo and Patrick Stickler. Brian McBride was the series editor for this earlier specification.

§ G. Substantive Changes

This section is non-normative.

§ G.1 Substantive changes between RDF 1.0 and RDF 1.1

This section is non-normative.

- The RDF 1.0 semantics defined simple interpretations relative to a vocabulary.
- In the RDF 1.0 semantics, IL was a total, rather than partial, mapping.
- The RDF 1.0 specification divided literals into 'plain' literals with no type and optional language tags, and typed literals. Usage has shown that it is important that every literal have a type. RDF 1.1 replaced plain literals without language tags by literals typed with the XML Schema `string` datatype, and introduced the special type `rdf:langString` for language-tagged strings. The full semantics for typed literals is given in Section 7. Literals and datatypes.
- In the RDF 1.0 specification datatype D-entailment was defined as a semantic extension of RDFS-entailment. In RDF 1.1 it was defined as a direct extension to basic RDF. This is more in conformity with actual usage, where RDF with datatypes is widely used without the RDFS vocabulary. If there is a need to differentiate from the RDF 1.0 terminology, the longer phrasing "simple D-entailment" or "simple datatype entailment" should be used rather than "D-entailment".
- RDF 1.0 specification defined the parameter D as a datatype map from IRIs to datatypes, i.e., as a restricted kind of interpretation mapping. As RDF 1.1 presumed that a recognized IRI identifies a unique datatype, this IRI-to-datatype mapping is globally unique and externally specified, so we can think of D as either a set of IRIs or as a fixed datatype map. Formally, the ***datatype map*** corresponding to the set D is the restriction of a D-interpretation to the set D. Semantic extensions which are stated in terms of conditions on datatype maps can be interpreted as applying to this mapping.

- In the RDF 1.0 specification, ill-typed literals were required to denote a value in IR, and D-unsatisfiability could be recognized only by using the RDFS semantics.
- In the 2004 RDF 1.0 semantics, LV was defined as part of a simple interpretation structure, and its definition in RDFS interpretations was a constraint.

§ G.2 Substantive changes since RDF 1.1

This section is non-normative.

- The major change between the RDF 1.1 and RDF 1.2 semantics is the addition of triple terms. Various parts of the semantics have been updated to handle triple terms.
- RDF entailment rule rdfD1a was added in RDF 1.2. This rule should have been included in RDF 1.1 when the two built-in datatypes (`xsd:string` and `rdf:langString`) were added to RDF entailment.
- `rdf:dirLangString` was added to the built-in datatypes.
- In RDF 1.1, `rdf:PlainLiteral` was described as an optional datatype that, "[if] [recognized](https://www.w3.org/TR/rdf11-mt/#dfn-recognize), ... *MUST* be interpreted to [denote] the datatype defined in *rdf:PlainLiteral: A Datatype for RDF Plain Literals (Second Edition)* [*RDF-PLAIN-LITERAL*]." `rdf:PlainLiteral` is not used elsewhere in the RDF documents, so the requirement to give it this particular semantics has been removed. It is recommended that `rdf:PlainLiteral` not be used in RDF.
- The appendix on RDF reification, containers, and collections has been removed because it had no semantic content. The vocabulary involved is described in the Legacy Vocabularies and RDF Collections sections of RDF Schema.

§ H. Index

§ H.1 Terms defined by this specification

class §9.

D-entails §7.2

D-interpretation §7.1

D-satisfiable §7.2

D-unsatisfiable §7.2

datatype map §G.1

denotation §4.

empty graph §4.

entailment regime §3.

entails recognizing D §7.2

extension §5.

generalized RDF (RDFS) closure §A.

GrdfD1 §A.

Grdfs14 §A.

ground RDF graph §4.

ground RDF term §4.

ground RDF triple §4.

ground triple term §4.

identify §4.

instance §4.

interpolation §5.3

interpretation §4.

invalid §5.2

L2V §7.

lean §4.

merge §4.1

merging §4.1

monotonic §3.

name §4.

proper instance §4.

proper subgraph §4.

RDF dataset merge §10.

RDF entail §8.1

RDF interpretation §8.

RDF unsatisfiable §8.1

RDF vocabulary §8.

rdfD1 §8.1.1

rdfD1a §8.1.1

rdfD2 §8.1.1

RDFS entails §9.2

RDFS interpretation §9.

RDFS vocabulary §9.

rdfs1 §9.2.1

rdfs10 §9.2.1

rdfs11 §9.2.1

rdfs12 §9.2.1

rdfs13 §9.2.1

rdfs14 §9.2.1

rdfs2 §9.2.1

rdfs3 §9.2.1

rdfs4 §9.2.1

rdfs5 §9.2.1

rdfs6 §9.2.1

rdfs7 §9.2.1

rdfs8 §9.2.1

rdfs9 §9.2.1

recognize §7.

refer to §4.

satisfiable §5.2

satisfiable recognizing D §7.2

satisfies §5.2

semantic extension §3.

set of facts §5.3

set of facts asserted by a graph §5.3

set of propositions §5.3

simple interpretation §5.

Skolemization §6.

standardize §4.1

subgraph §4.

substitution mapping §4.

unsatisfiable §5.2

valid §5.2

vocabulary §4.

§ H.2 Terms defined by reference

[*RDF12-CONCEPTS*] defines the following:

appears in

blank node

blank node identifiers

Datatypes

denotes

directional language-tagged strings

entail

equivalent

generalized RDF datasets

generalized RDF graphs

generalized RDF triples

generalized RDF triples, graphs, and datasets

ill-typed

inconsistent

IRI

isomorphic

language-tagged strings

lexical form

lexical-to-value mapping

literal

node

object type

predicate

Privacy Considerations

proposition

RDF dataset

RDF graph

RDF source

RDF term

RDF triple

RDF vocabularies

rdf:HTML

rdf:JSON

rdf:XMLLiteral

referent

reifying triple

Replacing Blank Nodes with IRIs

Security Considerations

strings

subject

symmetric RDF triples

triple term

value space

[*SWBP-XSCH-DATATYPES*] defines the following:

discussion

[*XML11*] defines the following:

Char production

[*XMLSCHEMA11-2*] defines the following:

xsd:string

§ I. References

§ I.1 Normative references

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 28 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 29 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RFC2119]

Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[XML11]

Extensible Markup Language (XML) 1.1 (Second Edition). Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau; John Cowan et al. W3C. 16 August 2006. W3C Recommendation. URL: https://www.w3.org/TR/xml11/

[XMLSCHEMA11-2]

W3C XML Schema Definition Language (XSD) 1.1 Part 2: Datatypes. David Peterson; Sandy Gao; Ashok Malhotra; Michael Sperberg-McQueen; Henry Thompson; Paul V. Biron et al. W3C. 5 April 2012. W3C Recommendation. URL: https://www.w3.org/TR/xmlschema11-2/

§ I.2 Informative references

[HORST04]

Extending the RDFS Entailment Lemma. Herman J. ter Horst. Springer, Berlin, Heidelberg. 2004. Conference Proceeding. URL: https://doi.org/10.1007/978-3-540-30475-3_7

[HORST05]

Completeness, Decidability and Complexity of Entailment for RDF Schema and a Semantic Extension Involving the OWL Vocabulary. Herman J. ter Horst. Journal of Web Semantics. 2005. Conference Proceeding. URL: https://dx.doi.org/10.2139/ssrn.3199251

[ISO24707]

Information technology — Common Logic (CL) — A framework for a family of logic-based languages. ISO/EIC. 2018-07. Published. URL: https://www.iso.org/standard/66249.html

[OWL2-OVERVIEW]

OWL 2 Web Ontology Language Document Overview (Second Edition). W3C OWL Working Group. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/owl2-overview/

[OWL2-PROFILES]

OWL 2 Web Ontology Language Profiles (Second Edition). Boris Motik; Bernardo Cuenca Grau; Ian Horrocks; Zhe Wu; Achille Fokoue. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/owl2-profiles/

[OWL2-SYNTAX]

OWL 2 Web Ontology Language Structural Specification and Functional-Style Syntax (Second Edition). Boris Motik; Peter Patel-Schneider; Bijan Parsia. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/owl2-syntax/

[RDF-MT]

RDF Semantics. Patrick Hayes. W3C. 10 February 2004. W3C Recommendation. URL: https://www.w3.org/TR/rdf-mt/

[RDF-PLAIN-LITERAL]

rdf:PlainLiteral: A Datatype for RDF Plain Literals (Second Edition). Jie Bao; Sandro Hawke; Boris Motik; Peter Patel-Schneider; Axel Polleres. W3C. 11 December 2012. W3C Recommendation. URL: https://www.w3.org/TR/rdf-plain-literal/

[RDF-TESTCASES]

RDF Test Cases. jan grant; Dave Beckett. W3C. 10 February 2004. W3C Recommendation. URL: https://www.w3.org/TR/rdf-testcases/

[RDF11-MT]

RDF 1.1 Semantics. Patrick Hayes; Peter Patel-Schneider. W3C. 25 February 2014. W3C Recommendation. URL: https://www.w3.org/TR/rdf11-mt/

[RDF12-INTEROP]

RDF 1.2 Interoperability. Pierre-Antoine Champin. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-interop/spec/

[RDF12-N-QUADS]

RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 20 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]

RDF 1.2 Schema. Dominik Tomaszuk. W3C. 26 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-TRIG]

RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 19 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[RIF-OVERVIEW]

RIF Overview (Second Edition). Michael Kifer; Harold Boley. W3C. 5 February 2013. W3C Working Group Note. URL: https://www.w3.org/TR/rif-overview/

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 26 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 29 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

[SWBP-XSCH-DATATYPES]

XML Schema Datatypes in RDF and OWL. Jeremy Carroll; Jeff Pan. W3C. 14 March 2006. W3C Working Group Note. URL: https://www.w3.org/TR/swbp-xsch-datatypes/

